Ejercicios

Ejercicio 8

Parte 1

1. Modelo

El problema se representa mediante un **grafo no dirigido y ponderado**:

- Cada vértice representa una zona.
- Cada arista representa un posible camino entre dos zonas.
- El peso de cada arista representa su distancia en kilómetros.

Se busca un **árbol abarcador mínimo**: una selección de caminos que conecte todas las zonas y cuya suma de distancias sea mínima.

Con 6 zonas se necesitan al menos **5 caminos** para conectarlas. El árbol abarcador tiene exactamente esos 5 caminos, sin ciclos. Para encontrarlo se puede utilizar **Prim**, incorporando en cada paso el camino más corto que conecte una zona ya incluida con otra todavía no incluida.

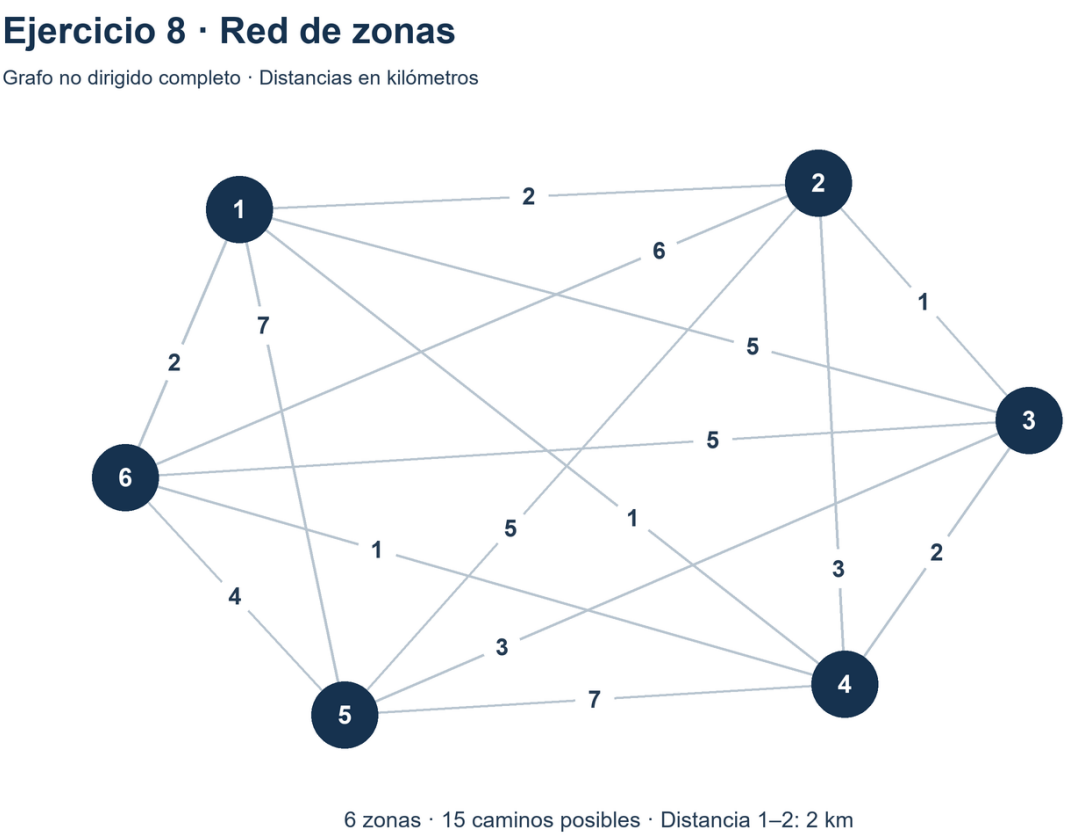
2. Lista de adyacencias

Cada paréntesis indica `(zona vecina, distancia)`:

Zona	Adyacencias
1	(2, 2), (3, 5), (4, 1), (5, 7), (6, 2)
2	(1, 2), (3, 1), (4, 3), (5, 5), (6, 6)
3	(1, 5), (2, 1), (4, 2), (5, 3), (6, 5)
4	(1, 1), (2, 3), (3, 2), (5, 7), (6, 1)
5	(1, 7), (2, 5), (3, 3), (4, 7), (6, 4)
6	(1, 2), (2, 6), (3, 5), (4, 1), (5, 4)

Cada arista aparece en las listas de sus dos extremos porque el grafo es no dirigido.

3. Grafo



4. Ejecución de Prim

Comenzamos en la **zona 1**. Si hay empate, elegimos la arista cuyo par de extremos, escrito de menor a mayor, sea primero en orden numérico.

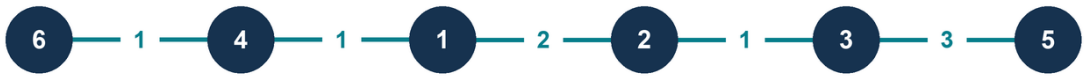
Paso	Zonas incluidas antes de elegir	Camino elegido	Distancia	Total
1	{1}	1–4	1	1
2	{1, 4}	4–6	1	2
3	{1, 4, 6}	1–2	2	4
4	{1, 2, 4, 6}	2–3	1	5
5	{1, 2, 3, 4, 6}	3–5	3	8

En el tercer paso, **1–2 y 4–3 empatan en 2 km**. Ambas elecciones permiten obtener una solución óptima.

El árbol obtenido es:

Ejercicio 8 · Árbol mínimo

Prim desde la zona 1 · Caminos seleccionados



5 caminos · Longitud total mínima: 8 km

Cada zona queda conectada con todas las demás, sin ciclos.

Resultado: construir 5 caminos, con una longitud total mínima de 8 km.

Parte 2

1. Pseudocódigo

Usamos una lista de vértices incluidos y otra de aristas seleccionadas. En cada paso buscamos la arista más barata que conecte un vértice incluido con uno que todavía esté afuera.

Precondiciones: grafo no dirigido, ponderado y con pesos comparables. Para obtener un árbol abarcador debe ser conexo. Si no lo es, el algoritmo lo informa.

```
ALGORITMO PRIM(grafo)

    incluidos ← colección vacía
    aristasSeleccionadas ← lista vacía
    costoTotal ← 0

    SI el grafo no tiene vértices ENTONCES
        DEVOLVER aristasSeleccionadas y costoTotal
    FIN SI

    inicio ← primer vértice del grafo
    agregar inicio a incluidos

    MIENTRAS cantidad de incluidos < cantidad de vértices del grafo HACER

        mejorArista ← NULO
        menorPeso ← infinito
        nuevoVertice ← NULO

        PARA CADA vértice u de incluidos HACER
            PARA CADA arista incidente en u HACER

                v ← el otro extremo de la arista

                SI v no pertenece a incluidos ENTONCES
                    SI mejorArista es NULO O peso de la arista < menorPeso ENTONCES
                        mejorArista ← arista
                        menorPeso ← peso de la arista
```

```
                        nuevoVertice ← v
                    FIN SI
                FIN SI

            FIN PARA
        FIN PARA

        SI mejorArista es NULO ENTONCES
            INFORMAR "El grafo no es conexo: no existe un árbol abarcador"
            TERMINAR sin devolver el resultado parcial como árbol abarcador
        FIN SI

        agregar mejorArista a aristasSeleccionadas
        agregar nuevoVertice a incluidos
        costoTotal ← costoTotal + menorPeso

    FIN MIENTRAS

    DEVOLVER aristasSeleccionadas y costoTotal

FIN ALGORITMO
```

En caso de empate puede elegirse cualquiera de las aristas mínimas. Para reproducir la resolución anterior, se recorren los vértices y sus vecinos en orden numérico.

El algoritmo no introduce ciclos: cada arista elegida incorpora un vértice nuevo. Cuando termina, tiene todos los vértices y exactamente **V − 1 aristas**.

2. Prueba con cuatro zonas

Tomamos las zonas 1, 2, 3 y 4 del ejercicio 8. Sus aristas son:

`1–2: 2; 1–3: 5; 1–4: 1; 2–3: 1; 2–4: 3; 3–4: 2.`

Paso	Incluidos antes del paso	Aristas que conectan con el exterior	Elección	Costo acumulado
1	{1}	1–2: 2; 1–3: 5; 1–4: 1	1–4	1
2	{1,4}	1–2: 2; 1–3: 5; 4–2: 3; 4–3: 2	1–2	3
3	{1,2,4}	1–3: 5; 2–3: 1; 4–3: 2	2–3	4

Resultado: árbol con aristas **1–4, 1–2 y 2–3**, y costo total **4 km**.

3. Casos de prueba

Caso	Datos	Resultado esperado
Grafo vacío	Sin vértices	Lista de aristas vacía y costo 0, según la convención adoptada.
Un vértice	A, sin aristas	Árbol sin aristas y costo 0.
Dos vértices conectados	A–B: 7	Una arista y costo 7.
Grafo que ya es un árbol	A–B: 2; B–C: 3	Ambas aristas y costo 5.
Ciclo con una arista costosa	A–B: 1; B–C: 2; A–C: 8	A–B y B–C; costo 3.
Empate de pesos	Triángulo con todas las aristas de peso 1	Cualquier par de aristas que conecte los tres vértices; costo 2.
Grafo desconectado	A–B: 1; C aislado	Informar que no existe árbol abarcador.
Peso cero	A–B: 0; B–C: 2; A–C: 5	A–B y B–C; costo 2.
Escenario de cuatro zonas	Datos de la prueba anterior	3 aristas y costo 4.
Escenario completo del ejercicio 8	Las seis zonas	5 aristas y costo 8.

Los pesos cero deben tratarse como pesos válidos, no como ausencia de arista.

Especificación de las pruebas en pseudocódigo:

```
PARA CADA caso de prueba HACER

    construir el grafo indicado
    ejecutar PRIM sobre ese grafo
```

```
    SI el caso es desconectado ENTONCES
        VERIFICAR que se informa que no existe árbol abarcador
    SINO
        VERIFICAR que el costo coincide con el esperado
        VERIFICAR que todas las aristas elegidas pertenecen al grafo
        VERIFICAR que la suma de sus pesos coincide con el costo informado

        SI el grafo tiene al menos un vértice ENTONCES
            VERIFICAR que se seleccionaron cantidadDeVertices − 1 aristas
            VERIFICAR que todos los vértices quedan conectados
            VERIFICAR que no hay ciclos
        SINO
            VERIFICAR que no se seleccionaron aristas
        FIN SI
    FIN SI

FIN PARA
```

En los casos con empates se verifica el costo y las propiedades del árbol; no se exige una única selección de aristas.

4. Póster

PRIM: CONECTAR TODAS LAS ZONAS CON EL MENOR COSTO

Problema

Conectar seis zonas mediante caminos de tierra, minimizando la distancia total construida.

Modelo

Grafo no dirigido y ponderado:

- Vértices: zonas.
- Aristas: caminos posibles.
- Pesos: distancias en kilómetros.

Objetivo

Encontrar un árbol abarcador mínimo: conecta todos los vértices, no tiene ciclos y minimiza la suma de los pesos.

Algoritmo de Prim

1. Elegir una zona inicial e incluirla en el árbol.
2. Buscar el camino de menor peso entre una zona incluida y una zona todavía no incluida.
3. Incorporar ese camino y la nueva zona.
4. Repetir hasta incluir todas las zonas.
5. Si quedan zonas afuera y no existe un camino que permita incorporarlas, informar que el grafo es desconectado.

Pseudocódigo resumido

```
incluidos ← {zona inicial}
caminos ← lista vacía
costo ← 0

MIENTRAS queden zonas sin incluir
    buscar la arista mínima entre incluidos y no incluidos

    SI no existe
        informar que no hay árbol abarcador
        terminar
    FIN SI

    agregar la arista a caminos
    agregar su extremo exterior a incluidos
    sumar su peso al costo
FIN MIENTRAS
```

Aplicación al ejercicio 8

Paso	Camino elegido	Distancia	Total
1	1–4	1 km	1 km
2	4–6	1 km	2 km

Paso	Camino elegido	Distancia	Total
3	1–2	2 km	4 km
4	2–3	1 km	5 km
5	3–5	3 km	8 km

Resultado: 6 zonas conectadas con 5 caminos y 8 km en total.

Pruebas esenciales

- Un vértice: cero caminos y costo cero.
- Dos vértices conectados: elegir su única arista.
- Grafo con ciclos: conectar sin formar ciclos.
- Empates: aceptar distintos árboles con el mismo costo mínimo.
- Grafo desconectado: informar que no existe árbol abarcador.
- Cuatro zonas del ejercicio: costo mínimo de 4 km.
- Seis zonas del ejercicio: costo mínimo de 8 km.

Comprobación final

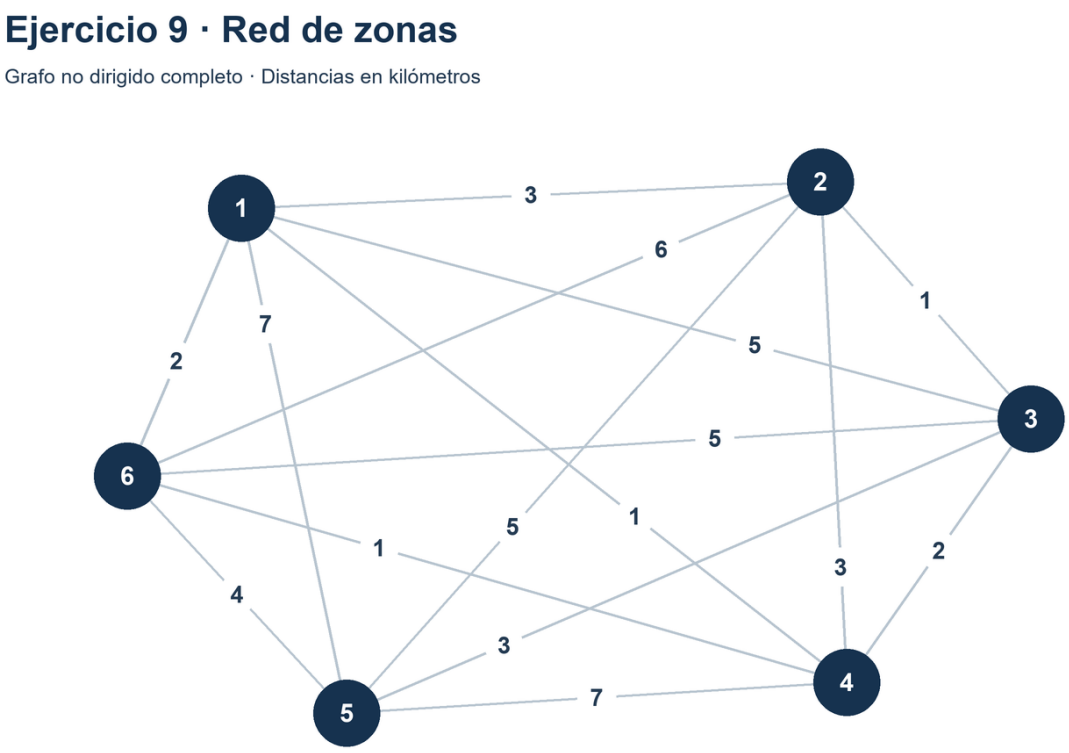
El resultado debe conectar todos los vértices, tener $V - 1$ aristas, no contener ciclos y alcanzar el costo mínimo esperado.

Ejercicio 9

Parte 1

1. Grafo

La diferencia respecto del ejercicio 8 es que **el camino 1–2 ahora mide 3 km**.



2. Ejecución de Kruskal

Kruskal ordena las aristas de menor a mayor peso y las incorpora siempre que **no formen un ciclo**.

Las aristas ordenadas por peso son:

Peso	Aristas
1	1–4, 2–3, 4–6
2	1–6, 3–4
3	1–2, 2–4, 3–5
4	5–6
5	1–3, 2–5, 3–6
6	2–6
7	1–5, 4–5

Al comienzo, cada zona es una componente separada: `{1}, {2}, {3}, {4}, {5}, {6}`.

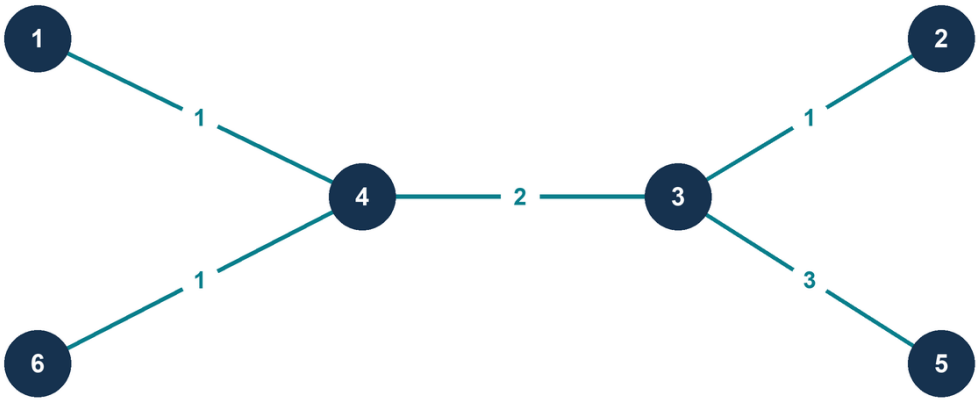
Paso	Arista evaluada	Decisión	Componentes resultantes	Total
1	1–4, peso 1	Aceptar	{1,4}, {2}, {3}, {5}, {6}	1
2	2–3, peso 1	Aceptar	{1,4}, {2,3}, {5}, {6}	2
3	4–6, peso 1	Aceptar	{1,4,6}, {2,3}, {5}	3

Paso	Arista evaluada	Decisión	Componentes resultantes	Total
4	1–6, peso 2	Rechazar: forma un ciclo	{1,4,6}, {2,3}, {5}	3
5	3–4, peso 2	Aceptar	{1,2,3,4,6}, {5}	5
6	1–2, peso 3	Rechazar: forma un ciclo	{1,2,3,4,6}, {5}	5
7	2–4, peso 3	Rechazar: forma un ciclo	{1,2,3,4,6}, {5}	5
8	3–5, peso 3	Aceptar	{1,2,3,4,5,6}	8

Terminamos al aceptar 5 aristas.

Ejercicio 9 · Árbol mínimo

Kruskal · Caminos seleccionados



5 caminos · Longitud total mínima: 8 km

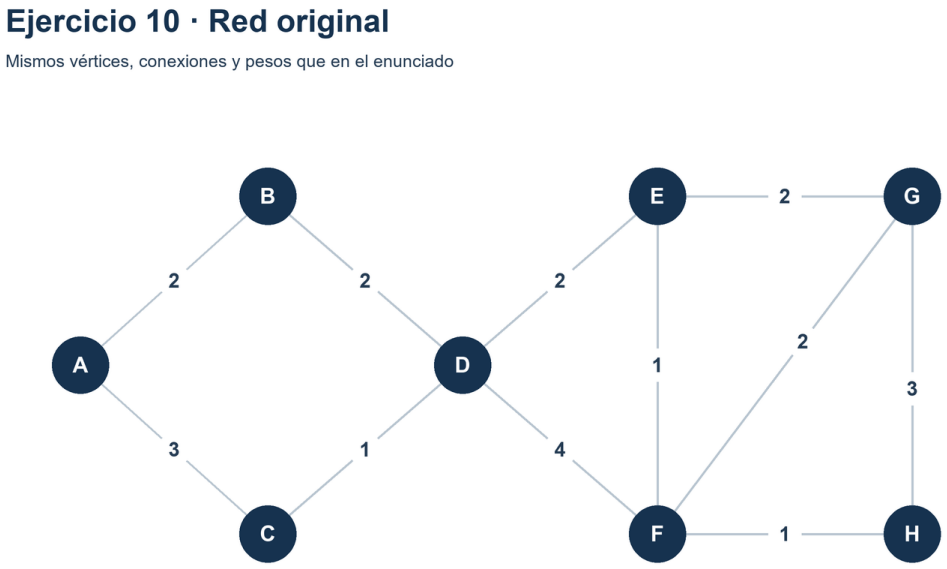
Cada zona queda conectada con todas las demás, sin ciclos.

Resultado: los caminos 1–4, 2–3, 4–6, 3–4 y 3–5 suman 8 km.

Ejercicio 10

Parte 1

Grafo original



Para BFS se ignoran los pesos y se comienza en E.

La búsqueda en amplitud, o **BFS**, visita primero los vecinos del origen y después los vértices que están a dos aristas, a tres, etc. **Los pesos del dibujo no intervienen en este recorrido.**

Como el enunciado no especifica un orden entre vecinos, utilizamos **orden alfabético**. Marcamos cada vértice como descubierto al incorporarlo a la cola, para evitar repeticiones.

Paso	Vértice procesado	Nuevos vértices descubiertos	Cola después del paso
Inicio	—	E	E
1	E	D, F, G	D, F, G
2	D	B, C	F, G, B, C
3	F	H	G, B, C, H
4	G	Ninguno	B, C, H
5	B	A	C, H, A
6	C	Ninguno	H, A
7	H	Ninguno	A
8	A	Ninguno	Vacía

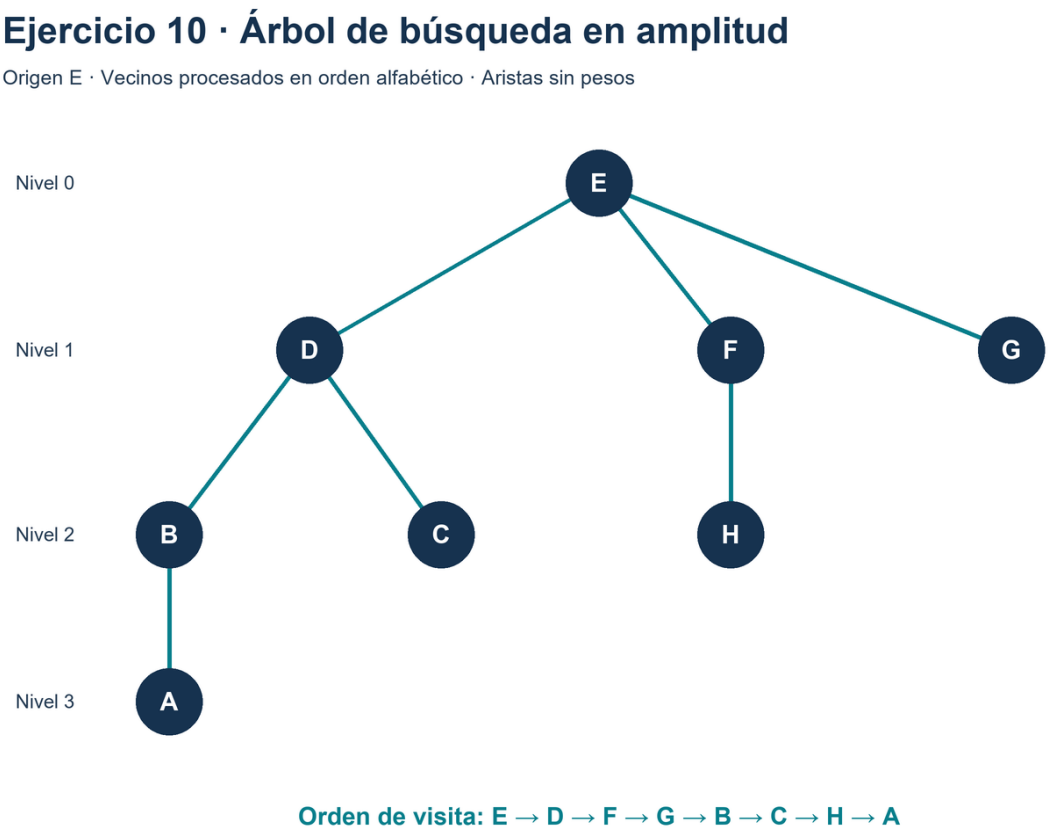
Orden de recorrido:

E → D → F → G → B → C → H → A

Las flechas de esta secuencia indican el orden de visita; no representan un camino que deba existir entre cada par consecutivo.

El árbol abarcador contiene las aristas por las que se descubrió cada vértice:

E–D, E–F, E–G, D–B, D–C, F–H y B–A.



Los niveles quedan así:

Nivel desde E	Vértices
0	E
1	D, F, G
2	B, C, H
3	A

Con otro orden de vecinos puede cambiar el orden de visita y alguna arista del árbol, pero **los niveles se mantienen**.

Parte 2

Precondiciones

- El grafo existe y permite consultar los vecinos de cada vértice.
- El vértice inicial pertenece al grafo.
- La cola y el conjunto de visitados comienzan vacíos.
- Para visitar todo el grafo desde un único origen, el grafo debe ser conexo. En caso contrario, se visita solamente la componente del origen.

Descripción en lenguaje natural

Se utiliza una cola: el primer vértice que entra es el primero en salir. Se incorpora el origen y se lo marca como visitado. Luego se extrae un vértice, se registra su visita y se incorporan sus vecinos no visitados.

Cada vecino se marca **al entrar en la cola**, evitando que se agregue más de una vez. También se registra desde qué vértice fue descubierto para construir el árbol abarcador.

```
ALGORITMO BUSQUEDA_EN_AMPLITUD(grafo, origen)

    SI origen no pertenece al grafo ENTONCES
```

```
        INFORMAR "El origen no pertenece al grafo"
        TERMINAR
    FIN SI

    visitados ← conjunto vacío
    cola ← cola vacía
    recorrido ← lista vacía
    aristasDelArbol ← lista vacía

    marcar origen como visitado
    encolar origen

    MIENTRAS la cola no esté vacía HACER

        actual ← desencolar
        agregar actual al recorrido

        PARA CADA vecino de actual, en orden alfabético, HACER

            SI vecino no está visitado ENTONCES
                marcar vecino como visitado
                agregar la arista actual–vecino a aristasDelArbol
                encolar vecino
            FIN SI

        FIN PARA

    FIN MIENTRAS

    DEVOLVER recorrido y aristasDelArbol

FIN ALGORITMO
```

Para el grafo del ejercicio, desde **E**, devuelve:

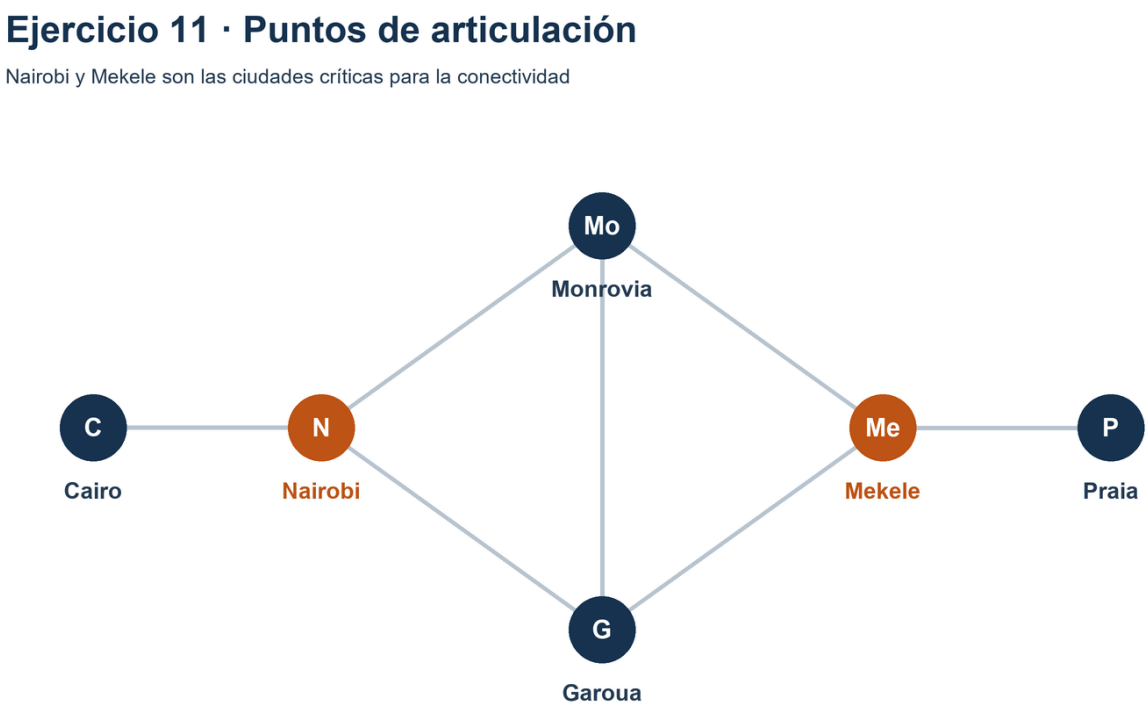
- **Recorrido:** E, D, F, G, B, C, H, A.
- **Árbol:** E–D, E–F, E–G, D–B, D–C, F–H, B–A.

Con listas de adyacencias, el recorrido requiere **O(V + E)** tiempo, sin contar un eventual ordenamiento de vecinos.

Ejercicio 11

Parte 1

1. Grafo



2. Resultado

Un **punto de articulación** es un vértice cuya eliminación, junto con sus aristas, aumenta la cantidad de componentes conexas.

En este grafo son **Nairobi y Mekele**:

Ciudad eliminada	Qué ocurre
Nairobi	Cairo queda aislada del resto de las ciudades.
Mekele	Praia queda aislada del resto de las ciudades.
Cairo	Las otras cinco ciudades siguen conectadas.
Praia	Las otras cinco ciudades siguen conectadas.
Monrovia	Nairobi y Mekele siguen conectadas a través de Garoua.
Garoua	Nairobi y Mekele siguen conectadas a través de Monrovia.

Aplicación del método mediante búsqueda en profundidad

Tomamos este orden de descubrimiento:

Nairobi → Cairo → Monrovia → Garoua → Mekele → Praia

Después de visitar Cairo se retrocede a Nairobi para continuar con Monrovia.

Las aristas del árbol de profundidad son:

- Nairobi–Cairo.
- Nairobi–Monrovia.
- Monrovia–Garoua.

- Garoua–Mekele.
- Mekele–Praia.

Las aristas adicionales **Garoua–Nairobi** y **Mekele–Monrovia** permiten regresar a antecesores.

Asignamos a cada ciudad un número de descubrimiento y un valor `low`. Este último indica el menor número de descubrimiento alcanzable desde su subárbol utilizando aristas del árbol y, como máximo, una arista de retorno a un antecesor.

Ciudad	Descubrimiento	low
Nairobi	1	1
Cairo	2	2
Monrovia	3	1
Garoua	4	1
Mekele	5	3
Praia	6	6

Aplicamos las condiciones:

- **Nairobi es la raíz del árbol y tiene dos hijos: Cairo y Monrovia.** Por eso es un punto de articulación.
- **Mekele tiene como hijo a Praia**, y `low(Praia) = 6 ≥ descubrimiento(Mekele) = 5`. Praia no tiene un camino alternativo hacia un antecesor de Mekele, por lo que Mekele también es un punto de articulación.
- Monrovia y Garoua cuentan con conexiones alternativas que evitan que su eliminación desconecte la red.

Resultado: las ciudades críticas para la conectividad son Nairobi y Mekele.

Parte 2

Se requiere un grafo no dirigido, con sus vértices y listas de vecinos disponibles. Inicialmente, todos los vértices deben estar sin visitar.

Se recorre el grafo en profundidad, registrando el orden de descubrimiento y el valor `low` de cada vértice. Al volver de cada rama se actualizan esos valores y se comprueba si algún hijo depende del vértice actual para conectarse con sus antecesores. La raíz se analiza por separado: es un punto de articulación si tiene más de un hijo en el árbol de profundidad.

Si el grafo es desconectado, se inicia otro recorrido desde cada vértice que haya quedado sin visitar. Con listas de adyacencias, el método requiere **O(V + E)** tiempo.

Parte 2

Precondiciones y operaciones utilizadas

El grafo es no dirigido y permite obtener sus vértices y consultar los vecinos de cada uno. Se supone un grafo simple, como el de la consigna.

En el pseudocódigo se utilizan las operaciones abstractas **“vértices del grafo”** y **“vecinos de un vértice”**, que deben corresponder a las operaciones disponibles en el TDA del curso.

Para cada vértice se registra:

- **Descubrimiento:** posición en la que fue visitado.
- **low:** menor número de descubrimiento alcanzable desde su subárbol usando aristas del árbol y, como máximo, una arista de retorno.
- **Padre:** vértice desde el cual fue descubierto.

Pseudocódigo detallado

```
ALGORITMO PUNTOS_DE_ARTICULACION(grafo)

    tiempo ← 0
    articulaciones ← conjunto vacío

    PARA CADA vértice v del grafo HACER
        visitado[v] ← FALSO
        padre[v] ← NULO
        descubrimiento[v] ← 0
        low[v] ← 0
    FIN PARA
```

```
    PARA CADA vértice v del grafo HACER
        SI visitado[v] es FALSO ENTONCES
            EXPLORAR(v)
        FIN SI
    FIN PARA

    DEVOLVER articulaciones

FIN ALGORITMO
```

El procedimiento auxiliar comparte las estructuras anteriores y el contador `tiempo`.

```
PROCEDIMIENTO EXPLORAR(u)

    visitado[u] ← VERDADERO
    tiempo ← tiempo + 1
    descubrimiento[u] ← tiempo
    low[u] ← tiempo
    cantidadHijos ← 0

    PARA CADA vecino v de u HACER

        SI visitado[v] es FALSO ENTONCES

            padre[v] ← u
            cantidadHijos ← cantidadHijos + 1

            EXPLORAR(v)

            low[u] ← mínimo entre low[u] y low[v]

            SI padre[u] no es NULO
               Y low[v] ≥ descubrimiento[u] ENTONCES
                agregar u a articulaciones
            FIN SI

        SINO

            SI v es distinto de padre[u] ENTONCES
                low[u] ← mínimo entre low[u] y descubrimiento[v]
            FIN SI

        FIN SI

    FIN PARA

    SI padre[u] es NULO Y cantidadHijos > 1 ENTONCES
        agregar u a articulaciones
    FIN SI

FIN PROCEDIMIENTO
```

Explicación

Se recorre el grafo en profundidad. Al regresar de un hijo se comprueba si su subárbol puede llegar a un antecesor del vértice actual sin pasar por este.

Si no puede, eliminar el vértice actual separaría ese subárbol del resto de la componente: por eso es un punto de articulación.

La raíz se analiza aparte: es un punto de articulación cuando tiene **más de un hijo en el árbol de profundidad**, no simplemente más de un vecino.

El conjunto evita devolver una ciudad varias veces. El recorrido exterior permite analizar también grafos desconectados.

Para la red del ejercicio, el resultado es **{Nairobi, Mekele}**. Con listas de adyacencias, el tiempo es **O(V + E)**.

Ejercicio 12

Punto 4

El número de Bacon es la **cantidad mínima de aristas** entre un actor y Kevin Bacon. Por eso, el método adecuado para calcularlo es una búsqueda en amplitud.

Las siguientes son redes ficticias de prueba. `K` representa a `Kevin_Bacon`; cada arista tiene como etiqueta una película.

Caso	Red de prueba	Consulta	Resultado esperado
El propio Kevin Bacon	Vértice K	K	0
Relación directa	K–A	A	1
Un intermediario	K–A–B	B	2
Tres vínculos	K–A–B–C	C	3
Dos rutas de distinta longitud	K–A–D y K–B–C–D	D	2
Dos rutas mínimas	K–A–D y K–B–D	D	2, independientemente de la ruta elegida
Grafo con ciclo	K–A, A–B, B–K	B	1; el recorrido debe terminar sin repetir visitas indefinidamente
Actor aislado	K y A, sin aristas	A	Informar que no hay conexión
Otra componente	K–A y B–C	C	Informar que no hay conexión
Actor inexistente	Solo existen K y A	X	Informar que el actor no pertenece al grafo
Relación no dirigida	Se carga la relación A–K	A	1, aunque K aparezca como segundo extremo
Consultas sucesivas	K–A–B	Primero B, después A	2 y 1; una consulta no debe alterar la siguiente

Para un actor existente pero sin conexión, puede adoptarse **−1** como valor de retorno. La consigna no fija ese valor: lo esencial es distinguir ese caso del número **0**, que corresponde únicamente a Kevin Bacon.

También conviene verificar la carga de los archivos:

Prueba de carga	Resultado esperado
Actor repetido	No crear dos vértices para la misma etiqueta.
Relación repetida	No duplicar la misma arista.
Relación con un actor ausente del archivo de actores	Detectar e informar la inconsistencia.
Película usada como etiqueta	Conservar su nombre en la arista correspondiente.

Los números concretos de **John_Goodman, Tom_Cruise, Jason_Statham, Lukas_Haas y Djimon_Hounsou** corresponden a la ejecución pedida en el punto 3. No pueden determinarse con la información disponible: dependen de las conexiones de los archivos `actores.csv` y `en_películas.csv`.

Aclaración: este PDF fue generado con ayuda de inteligencia artificial a partir de los ejercicios desarrollados previamente en papel, que fueron proporcionados para organizar y presentar las resoluciones de manera prolija.